

Reviewer Pack — Slice Rank of Layer-Variable Tensor Lower-Bounds Read-Twice ...

Entry f9268bf85309 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 01:14:26 UTC

Slice Rank of Layer-Variable Tensor Lower-Bounds Read-Twice BP Size for IP₂

Entry ID: f9268bf85309 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 01:14:26 UTC

1. Conjecture

Field A (mathematical branch): Slice rank / partition rank in additive combinatorics (Crooot-Lev-Pach 2016, Tao 2016, Naslund 2020) **Field B** (complexity object): Read-twice branching programs computing $IP_2 = \bigoplus_i x_i \cdot y_i$ (Borodin-Razborov-Smolensky 1993 read-twice barrier)

Statement:

For a read- k branching program P with vertex set $V(P)$, define the layer-variable tensor $T_P \in F_2^{\{|V| \times |V| \times n\}}$ by $T_P[u, v, j] := (\text{number of edges } u \rightarrow v \text{ labelled by a literal of } x_j) \bmod 2$, and let $\rho(P) := \text{slice_rank}_{\{F_2\}}(T_P)$. Conjecture (A) For every read-twice branching program P , $\rho(P) \leq 6 \cdot \log_2 |V(P)|$. Conjecture (B) For every (unrestricted) branching program P that computes IP_2 on n variables, $\rho(P) \geq n/8$; in particular any read-twice P for IP_2 satisfies $|V(P)| \geq 2^{\{n/48\}}$, and a single (P, ρ) pair refuting either inequality falsifies the claim.

Rationale (proposer's reasoning):

Slice rank is the F_2 invariant that drove the cap-set collapse: it measures how far a 3-tensor is from being decomposable along axis-aligned slabs, and read-twice forces the variable axis of T_P to be a sparse, low-multiplicity fiber pattern — exactly the regime where Tao-style slice arguments give logarithmic bounds. Conversely, IP_2 is the canonical bilinear obstruction, and any BP encoding it must spread the x_j -fibers of T_P across many slices because each x_j actually multiplies a distinct y_j , conjecturally producing a partition-rank-style anti-cap configuration with linear slice rank. The bridge is novel because slice rank has been deployed for cap sets, sunflower bounds, and matrix multiplication exponent, but never (to <5 papers) as a BP-size lower-bound invariant.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a86eb3994d9fa519

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds for each $n \in \{6, 8, 10, 12, 14, 16, 18, 20\}$ (read-twice BPs, $w \in \{2, 3, 4\}$), greedy slice-rank upper bound $\rho(P)$ must satisfy $\rho \leq 6 \cdot \lceil \log_2 |V(P)| \rceil$ on $\geq 27/30$ seeds per n ; and for canonical IP_2 BPs at $n \in \{4, 6, 8, 10, 12\}$, partition-rank lower bound must give $\rho \geq \lceil n/8 \rceil$ for every n .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - slice rank branching program lower bound - partition rank inner product mod 2 communication complexity - read-twice branching program IP2 tensor rank lower bound

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def matrix_multiply(A, B):
        m = len(A)
        n = len(B[0])
        p = len(B)
        C = [[0 for _ in range(n)] for _ in range(m)]
        for i in range(m):
            for j in range(n):
                for k in range(p):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def gaussian_elimination(A, b):
        m = len(A)
        n = len(A[0])
        augmented_matrix = [A[i] + [b[i]] for i in range(m)]

        for j in range(n):
            max_row = j
            for i in range(j+1, m):
```

```

        if abs(augmented_matrix[i][j]) > abs(augmented_matrix[max_row][j]):
            max_row = i

    augmented_matrix[j], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[j]

    pivot = augmented_matrix[j][j]
    for k in range(n+1):
        augmented_matrix[j][k] /= pivot

    for i in range(m):
        if i != j:
            factor = augmented_matrix[i][j]
            for k in range(n+1):
                augmented_matrix[i][k] -= factor * augmented_matrix[j][k]

    return [row[-1] for row in augmented_matrix]

def slice_rank(T):
    m, n, _ = len(T), len(T[0]), len(T[0][0])
    rank = 0
    while True:
        max_support = 0
        axis = -1
        v = [0] * n

        for i in range(3):
            support = 0
            if i == 0:
                for j in range(n):
                    for k in range(m):
                        if sum(T[k][j]) % 2 != 0:
                            support += 1
                            v[j] += 1
            elif i == 1:
                for k in range(m):
                    for j in range(n):
                        if sum(T[k][j]) % 2 != 0:
                            support += 1
                            v[k] += 1
            else:
                for j in range(n):
                    for k in range(m):
                        if T[k][j][k] % 2 != 0:
                            support += 1
                            v[j] += 1

            if support > max_support:
                max_support = support
                axis = i

        if max_support == 0:
            break

        rank += 1
        for j in range(n):
            T[j][j] -= v[j]

```

```

    return rank

def generate_read_twice_bp(n, w):
    V = [i for i in range(2*n)]
    edges = []
    for _ in range(w):
        for i in range(n):
            u = random.choice(V)
            v = random.choice(V)
            while v == u:
                v = random.choice(V)
            edges.append((u, v))
            V.remove(u)
            V.remove(v)

    T_P = [[[0 for _ in range(n)] for _ in range(2*n)] for _ in range(2*n)]
    for u, v in edges:
        for j in range(n):
            if random.choice([True, False]):
                T_P[u][v][j] += 1
            if random.choice([True, False]):
                T_P[v][u][j] += 1

    return T_P

def generate_ip2_bp(n):
    V = [i for i in range(2*n)]
    edges = []
    for i in range(n):
        u = random.choice(V)
        v = random.choice(V)
        while v == u:
            v = random.choice(V)
        edges.append((u, v))
        V.remove(u)
        V.remove(v)

    T_P = [[[0 for _ in range(n)] for _ in range(2*n)] for _ in range(2*n)]
    for u, v in edges:
        for j in range(n):
            if random.choice([True, False]):
                T_P[u][v][j] += 1
            if random.choice([True, False]):
                T_P[v][u][j] += 1

    return T_P

def partition_rank(T):
    m, n, _ = len(T), len(T[0]), len(T[0][0])
    rank = 0
    while True:
        max_support = 0
        axis = -1
        v = [0] * n

```

```

for i in range(3):
    support = 0
    if i == 0:
        for j in range(n):
            for k in range(m):
                if sum(T[k][j]) % 2 != 0:
                    support += 1
                    v[j] += 1
    elif i == 1:
        for k in range(m):
            for j in range(n):
                if sum(T[k][j]) % 2 != 0:
                    support += 1
                    v[k] += 1
    else:
        for j in range(n):
            for k in range(m):
                if T[k][j][k] % 2 != 0:
                    support += 1
                    v[j] += 1

    if support > max_support:
        max_support = support
        axis = i

if max_support == 0:
    break

rank += 1
for j in range(n):
    T[j][j] -= v[j]

return rank

n_values = [6, 8, 10, 12, 14, 16, 18, 20]
w_values = [2, 3, 4]
results = []

for n in n_values:
    for _ in range(30):
        T_P = generate_read_twice_bp(n, random.choice(w_values))
        rank = slice_rank(T_P)
        if rank > 6 * math.ceil(math.log2(2*n)):
            return {
                "metric_name": "slice_rank",
                "metric_value": rank,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": f"Read-twice BP with n={n} and rank {rank}"
            }
    results.append(rank)

for n in [4, 6, 8, 10, 12]:
    T_P = generate_ip2_bp(n)
    rank = partition_rank(T_P)
    if rank < n // 8:

```

```

        return {
            "metric_name": "partition_rank",
            "metric_value": rank,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"IP_2 BP with n={n} and rank {rank}"
        }
    results.append(rank)

    return {
        "metric_name": "slice_rank",
        "metric_value": sum(results) / len(results),
        "instances_tested": len(results),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    mean = sum(results) / len(results)
    std = (sum((x - mean)**2 for x in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r <= 6 * math.ceil(math.log2(2*n_values[0]))) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(r < n // 8 for n in [4, 6, 8, 10, 12]):
        first_failing_seed = seeds[results.index(next(r for r in results if r < n // 8))]
        print(f"RESULT: FALSIFIED counterexample=\"IP_2 BP\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ebc5334.py", line 231, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ebc5334.py", line 192, in run_trial
    T_P = generate_read_twice_bp(n, random.choice(w_values))
          ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ebc5334.py", line 104, in generate_read_twice_bp
    u = random.choice(V)

```

^^^^^^^^^^^^^^^^

```
File "/usr/lib/python3.12/random.py", line 347, in choice
    raise IndexError('Cannot choose from an empty sequence')
IndexError: Cannot choose from an empty sequence
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an `IndexError` in `generate_read_twice_bp` (empty sequence passed to `random.choice`) before producing any data, so neither the support nor falsification conditions were evaluated. | next: Fix `generate_read_twice_bp` to guarantee a non-empty vertex set V (e.g., enforce a minimum width and seed initial layers) before sampling, then re-run the 240-sample sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	157014
2	preregistration	claude_max	opus	0	0	8363
3	novelty	claude_max	opus	0	0	3541
4	novelty	claude_max	opus	0	0	9218
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	115721
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21312
7	judge	claude_max	opus	0	0	4431

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 319600 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/f9268bf85309.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f9268bf85309.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f9268bf85309.tar.gz` (if generated)